INFORME FINAL _ DESAFÍO 1 _ ESTEBAN MOSQUERA ROMERO.

Resumen

El presente informe analiza de manera exhaustiva un proyecto de Tetris por consola desarrollado en C++ con énfasis en manipulación de bits, memoria dinámica y diseño modular. El estudio se basa en el código fuente contenido en los archivos main.cpp, juego.cpp, tablero.cpp, pieza.cpp y consola.cpp, junto con sus encabezados asociados. A partir de la revisión del código se identificó que el tablero se implementa como una estructura dinámica respaldada por un bloque contiguo de bytes, mientras que las piezas se representan mediante máscaras de 16 bits que modelan matrices virtuales de 4 x 4. El sistema de rotación se construye con desplazamientos de bits y operaciones lógicas, la detección de colisiones se apoya en una comprobación hipotética de la pieza activa contra el tablero, la fijación de piezas se resuelve con operaciones OR a nivel de bit y la limpieza de filas se efectúa mediante inspección por bytes completos y desplazamiento descendente de la memoria.

El análisis demuestra que el proyecto adopta decisiones coherentes con un enfoque de bajo nivel, pues prioriza control explícito de memoria, economía de representación y eficiencia espacial. También se identifican fortalezas en la separación por módulos, en el diseño del ciclo principal de juego y en el uso consistente de operadores bitwise. Sin embargo, se reconocen riesgos académicos asociados con la portabilidad limitada a Windows, la ausencia de técnicas avanzadas como wall kicks, la dependencia de rand para la aleatoriedad y la gestión manual de memoria sin recursos modernos de la biblioteca estándar. En conjunto, el proyecto evidencia dominio de conceptos fundamentales de programación de sistemas y constituye una solución técnicamente sólida para un contexto universitario.

Palabras clave: C++, Tetris, manipulación de bits, memoria dinámica, consola, estructuras de datos.

Introducción

La implementación de Tetris en consola representa un ejercicio académico especialmente valioso porque nos obliga como estudiantes a traducir un problema visual y dinámico a una lógica algorítmica rigurosa. En este tipo de proyectos no basta con dibujar formas en pantalla: es necesario diseñar una estructura de datos compacta, un modelo de movimiento determinista, un mecanismo de validación de estados y una estrategia de actualización coherente del tablero. Además, cuando la consigna involucra memoria dinámica y operadores bitwise, el reto se desplaza desde la simple programación funcional hacia la construcción de una solución de bajo nivel que evidencie comprensión real de la relación entre abstracción, memoria y representación binaria.

El código analizado desarrolla un Tetris de consola orientado a Windows que utiliza una arquitectura modular. El punto de entrada se encuentra en main.cpp, la lógica general del juego se concentra en la clase Juego, la representación física del tablero se resuelve en el módulo tablero, la lógica geométrica de las piezas se define en pieza y las rutinas de interacción con la consola se encapsulan en consola. Esta separación favorece la legibilidad y permite estudiar cada responsabilidad de manera individual sin perder la visión global del sistema.

El objetivo de este informe es explicar cómo fue desarrollado mi proyecto a partir de su código fuente. Para ello se documenta la representación del tablero, la codificación de las piezas, la rotación, el sistema de colisiones, la fijación de bloques, la eliminación de filas, la detección de Game Over, las estructuras de datos utilizadas, la gestión de memoria dinámica y los operadores bitwise empleados.

Análisis del problema

El problema computacional que resuelve el proyecto consiste en simular un tablero rectangular donde piezas tetrominó descienden por gravedad, responden a entradas del usuario, colisionan con límites o bloques ya fijados y generan una reconfiguración del estado cuando se completa una fila. Aunque a nivel conceptual el juego parece sencillo, en realidad exige coordinar varios subsistemas: representación espacial, actualización temporal, interacción asíncrona, detección de eventos y persistencia del estado acumulado del tablero.

Desde la perspectiva algorítmica, el principal desafío radica en representar el tablero y las piezas con el menor costo posible sin sacrificar claridad. El proyecto opta por dos decisiones centrales. La primera es almacenar el tablero en memoria como una matriz binaria compactada en bytes. La segunda es modelar cada pieza con una máscara de 16 bits. Ambas elecciones reducen el consumo de memoria y permiten que la mayoría de las operaciones relevantes se formulen mediante desplazamientos y comparaciones bit a bit, lo cual se alinea de manera directa con los objetivos académicos del curso.

Otro desafío importante es la coexistencia de dos estados simultáneos: el tablero consolidado y la pieza activa. El tablero contiene exclusivamente piezas ya fijadas, mientras que la pieza actual existe como una superposición temporal que solo se integra al tablero cuando la gravedad o una colisión la inmovilizan. Esta separación obliga a validar movimientos y rotaciones de forma hipotética antes de modificar el estado permanente. En el código, dicha idea se materializa mediante llamadas reiteradas a ColisionPieza antes de alterar la posición o la forma de la pieza activa.

Finalmente, el proyecto debe resolver el problema del tiempo. En un juego de consola sin motor gráfico externo, el programador debe crear manualmente un loop principal que atienda

la entrada del usuario, redibuje la pantalla y administre la caída automática. El método Juego::Iniciar, en juego.cpp, cumple precisamente esta función al combinar lectura asíncrona de teclado, renderizado periódico y una gravedad controlada por acumulación de milisegundos.

Diseño de la solución

La solución sigue una organización modular sencilla y efectiva. El archivo main.cpp inicializa la semilla del generador aleatorio, limpia la consola, crea una instancia de Juego con dimensiones 16 x 20 e invoca el loop principal mediante el método Iniciar. En términos arquitectónicos, esta decisión convierte a la clase Juego en el coordinador del sistema, mientras que los demás módulos funcionan como servicios especializados.

Módulo	Responsabilidad principal	Elementos clave	Ubicación en el código
main	Inicialización general del programa	srand, system("cls"), creación de Juego, llamada a Iniciar	main.cpp
juego	Ciclo principal, entrada, gravedad, puntaje, Game Over	Juego, GenerarNuevaPieza, Dibujar, ManejarEntradaAsincrona, BajarPiezaGravedad	juego.h / juego.cpp
tablero	Modelo físico del tablero y	Tablero, CrearTablero, CeldaOcupada,	tablero.h / tablero.cpp

	operaciones persistentes	ColisionPieza, FijarPieza, LimpiarFilas	
pieza	Representación y rotación de tetrominós	MascaraPieza, PIEZAS_BASE, GenerarPiezaAleatoria, RotarDerecha, RotarIzquierda	pieza.h / pieza.cpp
consola	Acceso a la API de Windows para la interfaz textual	OcultarCursor, MoverCursor, LetraPresionada, ObtenerLetra, Dormir	consola.h / consola.cpp

El diseño no mezcla responsabilidades. Las rutinas de bajo nivel que manipulan la consola no contaminan la lógica geométrica de las piezas, y el almacenamiento del tablero queda aislado en una estructura específica. Esta modularidad beneficia la mantenibilidad del proyecto y facilita su explicación académica, pues permite justificar cada decisión de implementación con referencia a un archivo concreto.

Desde el punto de vista de la orientación a objetos, el núcleo del sistema es la clase Juego definida en juego.h. Sus atributos privados incluyen un puntero al tablero (m_tablero), la pieza activa (m_activa), el puntaje, el contador de líneas y el estado de fin del juego. Esta selección de atributos evidencia una modelación mínima pero suficiente del estado global. La clase no delega el tablero por valor, sino por puntero, porque el tablero se crea dinámicamente y su vida útil coincide con la de la instancia de Juego.

La construcción de la solución también muestra una intención clara de trabajar con datos compactos. En lugar de usar matrices de enteros o arreglos bidimensionales de booleanos para las piezas, el proyecto representa cada tetrominó con un entero sin signo de 16 bits. En lugar de reservar una matriz clásica para el tablero, se almacena cada fila como una secuencia de bytes. Estas elecciones reducen el tamaño del estado del juego y constituyen la base técnica sobre la que se apoyan las operaciones lógicas posteriores.

Representación del tablero

La representación del tablero se declara en tablero.h mediante la estructura Tablero, que contiene cuatro campos: ancho, alto, bytesPorFila y datos. Los dos primeros registran las dimensiones lógicas del espacio de juego; bytesPorFila especifica cuántos bytes se necesitan para almacenar una fila completa; y datos es un puntero a unsigned char que apunta a la memoria contigua donde se guarda el tablero completo. Esta estructura es especialmente relevante porque no se modela como una matriz explícita de celdas, sino como un bloque lineal de memoria comprimida por bits.

La creación del tablero ocurre en CrearTablero, en tablero.cpp, líneas 5 a 25. Allí se imponen restricciones de tamaño mínimo y se fuerza el ancho a ser múltiplo de ocho. Esta decisión no es arbitraria: al garantizar que cada fila contenga un número entero de bytes, el cálculo del índice físico de una celda se simplifica. Después se reserva memoria dinámica con new unsigned char[totalBytes], donde totalBytes equivale a bytesPorFila multiplicado por la altura. Finalmente, un bucle inicializa cada byte en cero, lo que significa que todas las celdas comienzan vacías.

La consulta puntual de una celda se implementa en CeldaOcupada, líneas 38 a 47 de tablero.cpp. Allí se traduce una coordenada lógica (x, y) a una dirección física en el arreglo

lineal datos. El byte correspondiente se calcula como y * bytesPorFila + x / 8, mientras que el bit interno dentro de ese byte se obtiene con 7 - (x % 8). Esta fórmula permite que la celda de la izquierda corresponda al bit más significativo y la de la derecha al menos significativo dentro del mismo byte, favoreciendo una lectura consistente cuando se analiza el contenido binario.

Una propiedad interesante del diseño es el tratamiento especial de coordenadas fuera de rango. Si x es menor que cero, si x supera el ancho o si y excede la altura, CeldaOcupada devuelve verdadero, lo que modela los bordes como zonas sólidas para efectos de colisión. En cambio, si y es negativo, la función devuelve falso. Gracias a esta decisión, las piezas pueden nacer por encima del área visible y entrar gradualmente al tablero sin generar colisiones prematuras. Esta política resulta clave para la experiencia de juego y para la lógica de aparición de nuevas piezas.

Representación de las piezas

Las piezas se definen en pieza.h mediante el alias typedef unsigned short MascaraPieza. El uso de unsigned short es coherente con la necesidad de almacenar exactamente 16 bits, uno por cada celda de una matriz virtual de 4 x 4. Sobre esta base también se define la estructura PiezaActiva, que contiene la forma binaria de la pieza y sus coordenadas x e y dentro del tablero.

En pieza.cpp, líneas 8 a 16, se encuentra el arreglo constante PIEZAS_BASE con siete máscaras hexadecimales: 0x0F00, 0x0660, 0x0E40, 0x06C0, 0x0C60, 0x0E20 y 0x0E80. Cada valor codifica uno de los siete tetrominós clásicos. Por ejemplo, la línea horizontal I usa el patrón 0x0F00, que en binario corresponde a 0000 1111 0000 0000. Eso significa que,

dentro de la cuadrícula 4 x 4, la segunda fila está completamente encendida. La pieza O se representa con 0x0660, que produce un bloque compacto de dos por dos.

Esta estrategia es técnicamente elegante porque evita almacenar matrices independientes para cada forma y cada rotación. La pieza se conserva en una sola variable entera y puede leerse bit a bit cuando se necesita dibujar, verificar colisiones o fijarla en el tablero. El propio método Juego::Dibujar, en juego.cpp, líneas 48 a 56, demuestra ese principio: si la celda de pantalla cae dentro del cuadrado envolvente de la pieza activa, se calcula qué bit de la máscara le corresponde y se decide si se imprime un carácter de bloque.

Desde una perspectiva académica, la representación por máscara tiene dos ventajas notables. Primero, obliga al estudiante a razonar en términos de codificación binaria, desplazamientos y extracción de información. Segundo, produce una implementación más compacta y alineada con la manipulación de bits, objetivo central del ejercicio. Frente a una solución basada en arreglos bidimensionales tradicionales, esta versión exhibe un mayor nivel de control sobre la información almacenada.

Implementación de las rotaciones

La rotación de piezas se implementa en pieza.cpp mediante las funciones RotarDerecha y RotarIzquierda. La primera recorre la cuadrícula virtual de 4 x 4 usando dos bucles anidados para las coordenadas y y x. En cada posición calcula el índice del bit original como 15 - (y * 4 + x). Si ese bit está encendido en la máscara original, se calcula la nueva posición según la transformación geométrica de una rotación de 90 grados a la derecha, expresada en el código como 15 - (x * 4 + (3 - y)). Finalmente, el bit correspondiente se activa en la nueva máscara utilizando el operador OR.

La pieza O constituye un caso especial tratado explícitamente en las líneas 27 a 29 de pieza.cpp y nuevamente en las líneas 55 a 56 para la rotación izquierda. Como un cuadrado es invariante por rotación, el código devuelve la máscara sin modificar. Esta decisión evita cómputo innecesario y muestra criterio algorítmico, pues reconoce que no todas las piezas requieren el mismo procesamiento.

La rotación izquierda se implementa de manera derivada: en lugar de escribir una nueva fórmula de transformación, el código aplica tres veces RotarDerecha. Este enfoque, presente en las líneas 54 a 61 de pieza.cpp, privilegia la reutilización y reduce el riesgo de errores al mantener una sola lógica de rotación elemental. Aunque no es la alternativa más rápida en términos absolutos, el costo es despreciable dado que la pieza solo contiene 16 bits y cada rotación recorre un espacio fijo de 4 x 4.

La validación de una rotación no se realiza dentro del módulo pieza, sino en Juego::ManejarEntradaAsincrona. Cuando el usuario presiona W o Q, la nueva máscara se calcula primero en la variable temporal testRot y luego se consulta ColisionPieza. Solo si la rotación hipotética no colisiona se actualiza m_activa.forma. Esta separación es conceptualmente correcta porque el módulo pieza conoce la geometría, mientras que el módulo juego decide si la transformación es compatible con el estado actual del tablero.

Implementación de las colisiones

La detección de colisiones se concentra en la función ColisionPieza, líneas 50 a 65 de tablero.cpp. Su lógica consiste en recorrer las 16 posiciones de la máscara 4 x 4 de la pieza y, por cada bit encendido, comprobar si la celda correspondiente del tablero está ocupada. La función no modifica el estado del juego; simplemente evalúa si una ubicación hipotética es

válida. Esta naturaleza predictiva es esencial porque permite comprobar desplazamientos laterales, descensos y rotaciones antes de ejecutarlos.

El comportamiento real de la colisión depende también de CeldaOcupada. Dado que esa función considera sólidos los bordes laterales, el fondo y los bloques ya fijados, ColisionPieza hereda automáticamente todas esas restricciones. En consecuencia, el mismo mecanismo sirve para impedir que la pieza atraviese paredes, salga del tablero o se superponga con bloques previamente asentados. Esta reutilización de una primitiva de consulta simplifica la lógica general y evita duplicidad de reglas.

En términos operativos, cada movimiento del usuario pasa por este filtro. Si se presiona A, el juego prueba la posición x - 1; si se presiona D, prueba x + 1; si se intenta bajar, prueba y + 1; y si se solicita una rotación, prueba la nueva máscara en la misma posición. Este patrón de validación puede verse en las líneas 89 a 115 de juego.cpp. El mérito de esta implementación radica en su uniformidad: todos los cambios de estado se aceptan solo después de una comprobación estructural contra el tablero.

Una limitación importante, pero totalmente comprensible en un proyecto universitario, es la ausencia de técnicas avanzadas de corrección de rotación, como wall kicks. En el Tetris moderno, si una rotación colisiona con una pared, el motor intenta pequeños desplazamientos laterales para hacerla válida. En este proyecto eso no existe: si la rotación colisiona, simplemente se rechaza. Académicamente, esta decisión no invalida la solución, pero sí conviene reconocerla como simplificación del modelo.

Fijación de piezas y persistencia del estado

La fijación de piezas se ejecuta cuando la gravedad intenta bajar la pieza activa y ColisionPieza detecta que la siguiente posición ya no es válida. En ese momento,

BajarPiezaGravedad, líneas 121 a 140 de juego.cpp, deja de mover la pieza y llama a FijarPieza. Esta llamada marca la transición conceptual entre la pieza temporal que está cayendo y el estado permanente del tablero.

La función FijarPieza, líneas 68 a 87 de tablero.cpp, recorre de nuevo la cuadrícula 4 x 4 de la máscara. Por cada bit activo calcula la coordenada real dentro del tablero, ubica el byte y el bit correspondientes y activa la celda mediante el operador OR: tablero->datos[byte_idx] |= (unsigned char)(1 << bit_offset). El uso de OR es apropiado porque nunca apaga información previa; únicamente consolida como ocupadas las posiciones donde la pieza deja bloques.

El método también filtra explícitamente las coordenadas reales para evitar escribir fuera del bloque reservado. Solo se procede si real_x y real_y caen dentro de los límites válidos. Esta protección es importante porque la pieza puede encontrarse parcialmente por encima del tablero en los primeros momentos de su aparición. Gracias a esta validación, solo las celdas visibles o ya ingresadas al campo de juego son persistidas en memoria.

Desde la perspectiva del diseño, la fijación es el punto donde el sistema deja de trabajar con hipótesis y actualiza definitivamente el estado global. A continuación se invoca LimpiarFilas y, después de resolver sus efectos, se genera una nueva pieza. Esta secuencia ordenada refleja correctamente el flujo canónico de Tetris: caída, bloqueo, limpieza, actualización de puntaje y creación de la siguiente pieza.

Limpieza de filas

La eliminación de filas completas se implementa en LimpiarFilas, líneas 90 a 123 de tablero.cpp. El algoritmo recorre todas las filas del tablero y parte del supuesto de que una fila está llena hasta demostrar lo contrario. Para verificarlo inspecciona cada byte que compone la fila y exige que todos sean iguales a 0xFF. Dado que el ancho del tablero se fuerza a ser

múltiplo de ocho, una fila completa equivale exactamente a una secuencia de bytes con todos sus bits en uno.

Cuando una fila está llena, el contador lineasLimpiadas se incrementa y se ejecuta un desplazamiento descendente: cada fila k recibe el contenido de la fila k - 1. Este proceso emula la gravedad posterior a la eliminación. Luego, la primera fila se rellena con ceros para representar el espacio vacío que aparece en la parte superior. La instrucción y--, ubicada al final del bloque de eliminación, obliga a reevaluar la misma posición vertical porque después del corrimiento ha llegado una nueva fila a ese índice. Este detalle evita omitir limpiezas consecutivas.

La estrategia es eficiente y coherente con la representación por bytes. En lugar de revisar celda por celda con comparaciones booleanas, el algoritmo se beneficia de la compactación binaria y reduce la condición de fila completa a una comparación byte a byte contra 0xFF. En un tablero de 16 columnas, por ejemplo, solo es necesario comprobar dos bytes por fila, lo que simplifica el coste de evaluación.

La limpieza de filas se integra además con el sistema de puntaje. En BajarPiezaGravedad, una vez fijada la pieza, el juego obtiene cuántas filas fueron limpiadas y suma puntaje según un esquema clásico: 40 puntos por una línea, 100 por dos, 300 por tres y 1200 por cuatro. Este detalle no solo mejora la jugabilidad, sino que evidencia que la lógica del proyecto ya no se limita a mover piezas, sino que incorpora una capa de reglas de juego consistente.

Detección de Game Over

La condición de Game Over se determina en GenerarNuevaPieza, líneas 22 a 31 de juego.cpp. Cada nueva pieza se genera aleatoriamente, se centra horizontalmente y se ubica inicialmente en y = -3, es decir, por encima del área visible del tablero. Sin embargo, la comprobación de

derrota no se realiza en y = -3, sino en la línea visible superior del tablero, usando ColisionPieza con coordenada vertical 0.

La lógica es la siguiente: si una pieza recién generada ya colisionaría al entrar al tablero visible, significa que la zona superior está ocupada y no existe espacio suficiente para continuar la partida. En ese caso se activa m_gameOver = true. Este criterio es razonable y está bien adaptado al hecho de que las piezas nacen parcialmente fuera de la zona de juego.

Además de esta condición principal, el usuario también puede finalizar la ejecución mediante la tecla Escape. En Juego::ManejarEntradaAsincrona, línea 113 de juego.cpp, el código ASCII 27 cambia el estado m_gameOver a verdadero. Aunque no corresponde a una derrota por saturación del tablero, sí constituye una salida controlada del loop principal.

En la práctica, el uso del mismo indicador booleano para derrota real y salida voluntaria simplifica el control del bucle. Esta unificación es válida para el alcance del proyecto, aunque en una versión más avanzada podría ser conveniente distinguir entre finalización manual y derrota efectiva mediante estados diferentes o una enumeración específica.

Estructuras de datos empleadas

El proyecto utiliza un conjunto reducido de estructuras de datos, pero cada una cumple un propósito técnico muy preciso. La primera es la estructura Tablero, que encapsula las dimensiones del campo de juego y el puntero a la memoria dinámica que almacena las celdas. La segunda es la estructura PiezaActiva, que agrupa la forma binaria de la pieza actual con sus coordenadas de posición. La tercera es el arreglo constante PIEZAS_BASE, que funciona como una pequeña tabla de consulta para las siete piezas base del juego.

A nivel de diseño orientado a objetos, la clase Juego opera como contenedor del estado global y orquestador del comportamiento. Internamente combina un puntero a Tablero, una instancia

de PiezaActiva y variables escalares enteras para puntaje y líneas completadas, además del indicador booleano de Game Over. Esta decisión evita la proliferación de clases innecesarias y se ajusta a un problema donde la complejidad reside más en la lógica de actualización que en la jerarquía de objetos.

También es importante destacar el uso de tipos primitivos con intención explícita. unsigned short se elige para garantizar una máscara de 16 bits; unsigned char se usa para trabajar byte a byte con la memoria del tablero; int se reserva para índices, dimensiones y contadores. Desde la perspectiva académica, este uso deliberado de los tipos refuerza la idea de que la representación de datos no es accidental, sino una parte central del diseño de la solución.

En síntesis, el proyecto no depende de contenedores de alto nivel como std::vector o std::array. En vez de ello, construye su propia representación con punteros, arreglos dinámicos y enteros compactos. Esta elección es plenamente coherente con una práctica universitaria orientada a comprender memoria dinámica y bitwise desde sus fundamentos.

Gestión de memoria dinámica

La memoria dinámica aparece de manera explícita en el módulo tablero. La función CrearTablero reserva primero un objeto Tablero con new Tablero y después un bloque contiguo de bytes con new unsigned char[totalBytes]. El proyecto no delega esta responsabilidad a contenedores automáticos ni a punteros inteligentes, lo cual significa que la liberación correcta de memoria se vuelve un deber del programador.

La destrucción se resuelve en DestruirTablero, líneas 28 a 33 de tablero.cpp, donde se invoca delete[] sobre tablero->datos y luego delete sobre la propia estructura. Esta secuencia es correcta y respeta la regla de simetría entre reserva y liberación. Posteriormente, el destructor de la clase Juego, líneas 17 a 20 de juego.cpp, llama a DestruirTablero(m_tablero). Por tanto,

mientras una instancia de Juego se cree y destruya normalmente, el tablero no debería producir fugas de memoria.

En términos de diseño, la elección de memoria contigua tiene un beneficio claro: favorece la localidad de referencia y simplifica el acceso secuencial durante operaciones como el recorrido por filas o el desplazamiento descendente después de una limpieza. La estructura también es liviana, pues solo almacena metadatos y un puntero, lo que hace que el costo real resida en el arreglo de bytes.

No obstante, desde una mirada crítica, la gestión manual abre riesgos potenciales. El código no implementa constructores de copia ni mecanismos de prevención frente a copias accidentales de Juego o de Tablero. En este proyecto particular el riesgo es bajo porque el tablero siempre se maneja por puntero y la clase Juego no se copia, pero conviene señalar que una versión más robusta podría beneficiarse de RAII con std::vector<unsigned char> o std::unique_ptr<unsigned char[]> para reforzar la seguridad de recursos.

Operadores bitwise utilizados y su función

El proyecto emplea operadores bitwise de manera transversal. No se trata de apariciones anecdóticas, sino de la base misma de la representación y transformación de la información. El análisis del código permite identificar los siguientes usos principales.

Operador	Ubicación principal	Propósito	Ejemplo funcional
<<	tablero.cpp, pieza.cpp, juego.cpp	Construir máscaras con un bit activo en una posición determinada	1 << bit_offset

&	tablero.cpp, pieza.cpp, juego.cpp	Consultar si un bit específico está encendido	actual & (1 << bit_viejo)
|=	tablero.cpp, pieza.cpp	Encender un bit en una estructura destino sin borrar los demás	rotada |= (1 << bit_nuevo)
%	tablero.cpp	Calcular la posición relativa del bit dentro de un byte	x % 8

El desplazamiento a la izquierda se usa para generar una máscara con un único bit activo. Esa máscara después puede compararse con un valor o fusionarse con otro. Este patrón aparece repetidamente cuando el código necesita extraer información de una pieza o encender una celda del tablero.

El operador AND se emplea para pruebas de pertenencia. Cuando una expresión como pieza & (1 << bit_viejo) produce un valor distinto de cero, el código sabe que la celda correspondiente de la máscara está ocupada. Esta técnica aparece tanto en el renderizado como en la rotación, en las colisiones y en la fijación de piezas.

El operador OR, especialmente en su forma de asignación compuesta |=, se utiliza para construir nuevas máscaras y para fijar definitivamente bloques en el tablero. En la rotación, permite transferir bits a la pieza rotada. En la fijación, permite fusionar la pieza con la memoria persistente del tablero sin alterar otros bloques ya existentes.

Gracias a estos operadores, el proyecto consigue resolver un problema visual utilizando exclusivamente aritmética binaria y direccionamiento de memoria. Este aspecto constituye una de las fortalezas académicas más evidentes del trabajo.

Algoritmos implementados y flujo de ejecución

El flujo global del programa comienza en main.cpp. Allí se inicializa la semilla de aleatoriedad con srand(time(NULL)), se limpia la consola con system("cls"), se crea el objeto Juego y se llama a Iniciar. Esta secuencia establece el contexto de ejecución y delega el control total al motor del juego.

Una vez dentro de Juego::Iniciar, se genera la primera pieza y se entra en un bucle while que se mantiene hasta que m_gameOver sea verdadero. En cada iteración se ejecutan tres operaciones principales: procesar entrada de teclado, dibujar el estado actual y administrar el tiempo de caída. El método Dormir(30) fija una cadencia aproximada de 33 cuadros por segundo, mientras que un acumulador de milisegundos controla cuándo debe activarse la gravedad.

La gravedad se implementa como una función del número de líneas completadas. La variable velocidadGravedad arranca en 600 milisegundos y disminuye en diez unidades por cada línea completada, con un límite inferior de 100 milisegundos. Esta decisión agrega progresión de dificultad al juego y demuestra que el autor no se limitó a una simulación estática, sino que incorporó una mecánica escalable.

El método Dibujar sobrescribe la consola desde la posición 0,0 mediante MoverCursor y evita el parpadeo que generaría limpiar toda la pantalla en cada fotograma. Además, muestra el puntaje, el número de líneas, el tablero delimitado por paredes laterales y piso, y los controles

disponibles. Así, la interfaz textual cumple la doble función de representar el estado y comunicar la interacción al usuario.

Desde el punto de vista algorítmico, el proyecto combina una estrategia de simulación discreta con validación preventiva. Ninguna modificación del estado se realiza sin verificar previamente su viabilidad. Esta disciplina puede observarse en los movimientos laterales, las rotaciones y el descenso. Por esa razón, el comportamiento general del juego resulta consistente incluso con una implementación relativamente compacta.

Problemas afrontados y decisiones de desarrollo

El primer problema afrontado por el proyecto fue cómo representar piezas y tablero sin recurrir a matrices tradicionales. La solución adoptada consistió en traducir el espacio de juego a bits. Esto exige un razonamiento más abstracto, pero produce un diseño mucho más alineado con la temática del curso. La decisión de usar una máscara de 16 bits para las piezas y una secuencia contigua de bytes para el tablero evidencia una intención deliberada de trabajar cerca del nivel de máquina.

El segundo problema fue la sincronización entre entrada del usuario y gravedad automática. En un entorno de consola, el uso de lectura bloqueante habría detenido la caída de las piezas cada vez que el programa esperara una tecla. El módulo consola resuelve esta dificultad mediante _kbhit y _getch, encapsulados en LetraPresionada y ObtenerLetra. Así, el juego puede consultar si existe una entrada disponible sin detener el ciclo principal.

Otro reto importante fue evitar el parpadeo de la pantalla. En lugar de limpiar toda la consola en cada iteración, el código reposiciona el cursor al origen con MoverCursor y vuelve a escribir el tablero completo. Esta técnica, complementada con la ocultación del cursor, ofrece una experiencia visual mucho más estable y profesional para un proyecto por consola.

Finalmente, el desarrollo obligó a resolver correctamente la lógica de limpieza de filas y de generación de nuevas piezas. Ambos puntos son delicados y me dierón bastante dificil porque si la pieza se fija antes de tiempo, si la fila no se recorre otra vez tras una eliminación o si el tablero no se interpreta bien cuando la pieza nace parcialmente fuera del área visible, el juego se vuelve inconsistente. El código muestra que estas situaciones fueron contempladas mediante validaciones específicas y control cuidadoso de índices.

Evolución del proyecto a partir de la estructura observada

Aunque el repositorio entregado no contiene un historial de versiones, la estructura final del código permite inferir una evolución progresiva del proyecto. Es razonable sostener que el desarrollo inició con una representación base del tablero y una pieza móvil, y posteriormente se añadieron mecanismos de colisión, fijación, limpieza de filas, puntaje y control de velocidad. La actual separación en módulos sugiere un proceso de refinamiento donde la lógica fue extraída paulatinamente a funciones y archivos especializados.

La presencia de comentarios explicativos en casi todos los módulos indica además una fase de maduración orientada a la sustentación. No solo se implementó la lógica, sino que se dejó trazabilidad conceptual de por qué se eligió una determinada estrategia. Por ejemplo, el tablero se justifica por su contigüidad en memoria, la rotación del cuadrado se optimiza como caso especial y la lectura de teclado se explica desde la necesidad de no bloquear el bucle principal.

Asimismo, la forma en que se combina un enfoque procedural en los módulos de apoyo con una clase controladora en juego.h revela una evolución hacia una arquitectura más ordenada. En lugar de concentrar toda la lógica en main.cpp, el proyecto distribuye responsabilidades y

deja al método Iniciar como punto central del comportamiento. Esa decisión mejora la escalabilidad y facilita la argumentación académica del diseño.

En suma, el código final refleja un proyecto que no se quedó en una prueba mínima, sino que alcanzó un grado apreciable de completitud funcional. Existen claramente un modelo de datos, un motor de actualización, una interfaz de salida y un conjunto de reglas de juego enlazadas de forma coherente.

Fortalezas técnicas del proyecto

La primera fortaleza técnica es la representación compacta y eficiente de la información. Modelar piezas con 16 bits y el tablero con bytes contiguos demuestra dominio de compresión lógica y de direccionamiento manual de memoria. Esta característica diferencia el trabajo de soluciones más elementales basadas en matrices de enteros.

La segunda fortaleza es la coherencia modular. Los archivos cumplen responsabilidades distinguibles y el código evita mezclar la manipulación de consola con el tratamiento geométrico de las piezas o con la memoria del tablero. Esta separación mejora la comprensión y facilita tanto el mantenimiento como la exposición oral.

La tercera fortaleza radica en el uso consistente de validación hipotética antes de modificar el estado. La presencia de ColisionPieza como función central para movimientos, rotaciones y descenso garantiza uniformidad lógica y reduce errores de sincronización. Es una decisión de diseño correcta y elegante.

La cuarta fortaleza es la integración de varios conceptos avanzados en una sola aplicación funcional: memoria dinámica, operadores bitwise, estructuras personalizadas, encapsulación, temporización, entrada asíncrona y renderizado textual. El proyecto no demuestra solo conocimiento aislado de cada tema, sino su aplicación coordinada en una solución completa.

Finalmente, el proyecto presenta una implementación suficientemente sólida para ser sustentada como trabajo universitario serio. El uso de comentarios, la elección de nombres significativos y la claridad del flujo principal aportan valor pedagógico y hacen visible el razonamiento detrás del desarrollo.

Debilidades, limitaciones y riesgos académicos

La principal limitación técnica es la dependencia explícita de Windows. El módulo consola incluye windows.h y conio.h, y funciones como Sleep, _kbhit y _getch no son portables a otros sistemas operativos. Esto no invalida el proyecto, pero sí restringe su ejecución y conviene reconocerlo en una sustentación como una decisión contextual, no como una solución universal.

Otra limitación es la ausencia de técnicas avanzadas de jugabilidad presentes en implementaciones modernas de Tetris. No hay vista previa de la siguiente pieza, sistema de hold, hard drop, wall kicks ni nivelado formal más allá del ajuste de gravedad. Desde un enfoque académico esto no representa un error, pero sí delimita el alcance funcional del proyecto frente a un motor completo.

También se observa el uso de rand para generar piezas. Aunque suficiente para un proyecto universitario básico, este generador no ofrece distribución óptima ni control fino del azar. Una versión más robusta podría emplear <random> y un mecanismo de bolsa de siete piezas para evitar repeticiones poco equilibradas.

Desde la perspectiva de la ingeniería de software, la gestión manual de memoria es correcta pero frágil en comparación con alternativas modernas. El proyecto asume que la instancia de Juego no se copia y que la destrucción se producirá normalmente. En aplicaciones más

complejas, esta estrategia debería reforzarse mediante RAII y encapsulación del recurso dinámico.

Por último, el uso de system("cls") en main.cpp y system("pause>nul") al final del programa introduce dependencia del entorno y llamadas al sistema que suelen desaconsejarse en código más profesional. En este caso su empleo es comprensible por el contexto de consola en Windows, pero sigue siendo pertinente mencionarlo como una solución práctica más que ideal.

Conclusiones

El análisis del código fuente permite concluir que el proyecto de Tetris por consola en C++ fue desarrollado con una orientación de bajo nivel claramente intencional. La representación del tablero en un bloque contiguo de bytes y de las piezas en máscaras de 16 bits constituye el eje técnico del sistema y se articula de manera coherente con la detección de colisiones, la rotación, la fijación de bloques y la limpieza de filas.

Esta solución demuestra comprensión efectiva de memoria dinámica, operaciones bitwise, estructuras de datos personalizadas y organización modular del código. Además, el motor del juego integra correctamente entrada asíncrona, renderizado textual y temporización de gravedad, logrando una experiencia funcional y estable dentro de las limitaciones de una aplicación de consola.

Entre las mayores fortalezas del trabajo destacan la economía en la representación de datos, la uniformidad del mecanismo de validación de movimientos y la claridad de la separación por módulos. Estas características le otorgan al proyecto valor académico y evidencian que no se trata de una implementación improvisada, sino de una propuesta razonada desde la arquitectura y la eficiencia.

Al mismo tiempo, se identifican limitaciones reales en portabilidad, en técnicas avanzadas de jugabilidad y en robustez moderna de gestión de recursos. Sin embargo, dichas limitaciones no eclipsan el mérito principal del proyecto: traducir con éxito un problema clásico de videojuegos a una implementación universitaria que expone de forma tangible conceptos fundamentales de C++ y manipulación binaria.

Referencias

Deitel, P. J., & Deitel, H. M. (2020). C++ how to program (10th ed.). Pearson.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

ISO/IEC. (2020). Programming languages—C++. ISO.

Stroustrup, B. (2013). The C++ programming language (4th ed.). Addison-Wesley.

Código fuente del proyecto analizado: main.cpp, juego.cpp, tablero.cpp, pieza.cpp, consola.cpp y sus archivos de cabecera correspondientes.